

ProjectDAWIY : 自分好みの DAW 環境を DIY する 共通プラットフォームの構築

山崎 承太郎^{1,a)} 平山 心^{2,b)} 橋田 光代^{1,c)}

概要：ユーザが自身の制作スタイルに合わせて必要な機能を自ら取捨選択し、構築できる「DIY (Do It Yourself)」の思想を取り入れた新しい DAW のコンセプトとして「DAWIY」を提案する。DAWIY の目的は、柔軟な GUI、軽量の動作、そして何よりも高い拡張性を備えたクロスプラットフォームな音楽制作環境の実現である。この要件を満たすため、本研究では Web 技術 (HTML, JavaScript, TypeScript, CSS) を開発の基盤として採用した。

1. はじめに

現代の音楽制作において、Digital Audio Workstation (DAW) は中心的な役割を担っている。市販の DAW は多機能かつ高性能であるが、その一方で、必ずしも全てのユーザの特定のニーズやワークフローに合致しているとは限らない。また、機能が豊富であるために動作が重く、低スペックな PC では快適な操作が困難な場合がある。さらに、ユーザが独自に機能を拡張したり、不要な機能を削除したりするカスタマイズの自由度は極めて低いのが現状である。

このような背景から、本研究では、ユーザが自身の制作スタイルに合わせて必要な機能を自ら取捨選択し、構築できる「DIY (Do It Yourself)」の思想を取り入れた新しい DAW のコンセプトとして「DAWIY」を提案する [1]。DAWIY の目的は、柔軟な GUI、軽量の動作、そして何よりも高い拡張性を備えたクロスプラットフォームな音楽制作環境の実現である。この要件を満たすため、本研究では Web 技術 (HTML, JavaScript, TypeScript, CSS) を開発の基盤として採用した。

2. 設計思想

DAWIY では、色や配置といった UI や、機能など、なるべく全てをユーザが自由にカスタマイズできるようにす

る。しかしながら、これまで DAW を触ったことがないような未経験者にとっても扱いやすく理解しやすいようにしたいため、元々備わっている機能は少なくし、UI はシンプルに、直感的に作る。比較的ニッチな機能や、DAW を使い慣れたユーザにしか使いこなせないような機能は、後述する DAWIY プラグインによって、ユーザごとに個別に機能を導入できるようにする。

3. システムの基盤

この章では、DAWIY を構成する機能や、そのカスタマイズ性について述べる。まず、DAWIY は、WAM-Studio[2] を元に開発している。WAM-Studio とは、Web Audio Modules (WAMs) [3] という、Web ベースの DAW において VST (Virtual Studio Technology) プラグインのように音源やエフェクト (FX) を追加するために作られた共通規格を用いて開発された OSS の Web DAW である。

3.1 基本機能

この項では、DAWIY に予め実装されている、DAW として必須と思われる機能について述べる。

3.1.1 オーディオ・MIDI 入出力の制御

設定ウィンドウからデバイスに登録されているオーディオの再生デバイス、録音デバイス、MIDI の入力デバイス、出力デバイスをそれぞれ選択することができる。

3.1.2 再生、録音

再生ボタンを押すことで楽曲上に配置されたオーディオデータを再生したり、MIDI データを演奏して音を鳴らすことができる。MIDI データの演奏に際しては専ら WAMs

¹ 福知山公立大学情報学部

² 関西学院大学工学部

^{a)} 32545100@fukuchiyama.ac.jp

^{b)} ifp34362@kwansei.ac.jp

^{c)} hashida-mitsuyo@fukuchiyama.ac.jp

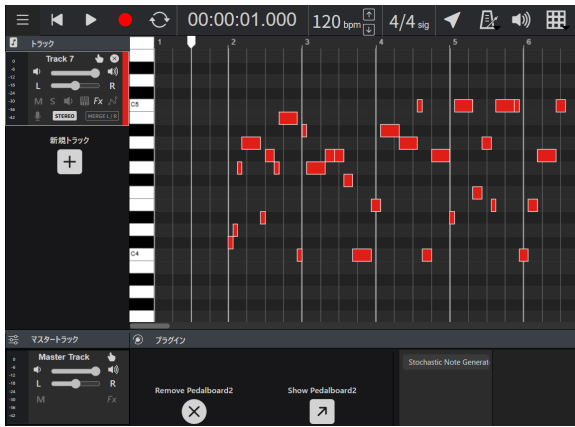


図 1: ピアノロール

を利用することになる。

3.1.3 トラック

複数のトラックを追加・削除することができる。各トラックで音声や MIDI をそれぞれ配置することができる。

3.1.4 ピアノロール

MIDI ノートを視覚的に配置・編集できるピアノロール画面を実装した (図 1)。具体的には、マウス操作によるノートの追加・削除、再生位置を示す再生バーの表示、ノートの長さの伸縮、複数ノートの範囲選択、および基本的なキーボードショートカットなどの機能を実現した。

3.2 便利機能

この項では、DAWIY に予め実装されている、あると便利な機能について述べる。

3.2.1 dawproject 形式での入出力

他の DAW との互換性を確保するため、dawproject 形式のプロジェクトファイルを読み込み、DAW 内に展開するインポート機能を実装した。逆に、本 DAW 上で作成したシーケンス情報を .dawproject ファイルとして書き出すエクスポート機能も実装した。これにより、Studio One や Cubase といった他の主要 DAW とのプロジェクトデータの交換が可能となる。

3.2.2 オートメーション

楽曲の進行に合わせて刻々とパラメータを変化させることができるオートメーション機能を実装した。オートメーションレーンはパラメータの数に合わせて動的に増減することが可能である。

3.2.3 ベロシティ

ピアノロール画面の下部に、各ノートのベロシティの可視化、また操作が可能なベロシティレーンを実装した。

3.3 どこで DIY できるのか

画面左上のハンバーガーメニューから見ることで設定ウィンドウにて、ショートカットキーを自由に変更したり、ハンバーガーメニューに出るメニューの管理を行う

ことができる。また、各 GUI の色も細かく設定したり、プリセットを利用することで一括でカラーテーマを変更することも可能である。それぞれの枠はドラッグアンドドロップで配置を自由に入れ替えることができる。このように、画面の見た目をそれぞれのユーザーにとってなるべく思い通りになるように DIY することができる。そして、それ以外にも、以下に述べる DAWIY プラグインを利用することで、より機能面に踏み込んだカスタマイズや、大幅な UI の変更・拡張を実現する。

4. DAWIY プラグイン

DAWIY プラグインは、VST プラグインではカバーできない、DAW の入出力・UI・操作等に介入する処理を拡張するための独自のプラグインシステムである。

4.1 種類と適用範囲

DAWIY プラグインは、AviUtl 及び AviUtl2[4] のプラグインを参考にし、プラグインを下記のカテゴリに分類する。プラグインによっては、複数のカテゴリを横断する機能を保有することも可能である。また、下記のいずれかにプラグインとして実装できそうなものであっても、その機能が DAWIY ユーザ全体に必要である、と判断される場合には、プラグインとして配布するよりもプルリクエストを出すなどして DAWIY 本体に取り込むことを推奨する。

- **Generator (生成)** : ノート、オーディオ、パターンを生成して楽曲に追加するプラグイン。
- **Modifier (加工)** : 既存のノートやオーディオデータを選択・加工するプラグイン。
- **Analysis (解析)** : 楽曲情報の可視化・分析を行うプラグイン。
- **IO (入出力)** : 外部ファイル形式の読み込み・書き出し機能を追加するプラグイン。
- **UI (ユーザインターフェース)** : DAW の全体的な外観の変更や、操作性の拡張を行うプラグイン。

これらのカテゴリは IDawiyPlugin インターフェース内で定義されており、プラグイン実装時に指定する。

4.2 全体管理

DAWIY プラグインは、public/src/DawiyPlugins ディレクトリ以下に配置された TypeScript ファイル (.ts) として管理される。システムはこのディレクトリを自動的にスキャンし、有効なプラグインファイルを検出してロードする仕組みとなっている。プラグインごとの設定やメタデータは、必要に応じて plugin.json 等の形式で管理されることもあるが、基本的には TypeScript のクラス定義内に記述されたメタデータ (ID, 名前, 説明など) が使用される。また、DAW 内部のパッケージマネージャ (図 2) より、登録されている DAWIY プラグインのインストール・アンイ



図 2: DAWIY パッケージマネージャー



図 3: クリエイターツール

ストール, また有効化・無効化が可能である. エンドユーザは, パッケージマネージャからインストールボタンを押すか, 事前にダウンロードした zip ファイル等を選択するだけで自動的にプラグインをインストールすることができるため, 内部のディレクトリ構造を把握する必要が無い.

4.3 作成方法

DAWIY プラグインの開発は, TypeScript を用いて行われる. 全てのプラグインは, 基底クラスである `DawiyPluginBase` を継承し, `@DAWIYPlugin` デコレータを付与したクラスとして定義する必要がある. 基本的な実装の流れは以下の通りである.

- (1) `DawiyPluginBase` クラスを継承したクラスを作成する.
- (2) クラスプロパティとして, 一意な ID, 名称, 説明などを定義する.
- (3) 必要に応じて, 初期化処理を行う `onInit` メソッド, UI を描画する `render` メソッド, 有効化・無効化時の処理を行う `onActivate` / `onDeactivate` メソッドをオーバーライドして実装する.

プラグイン内からは, `HostAPI` を通じて DAW のコア機能 (UI 操作, ファイルシステムアクセス, トラックデータの操作など) に安全にアクセスすることが可能である. ファ

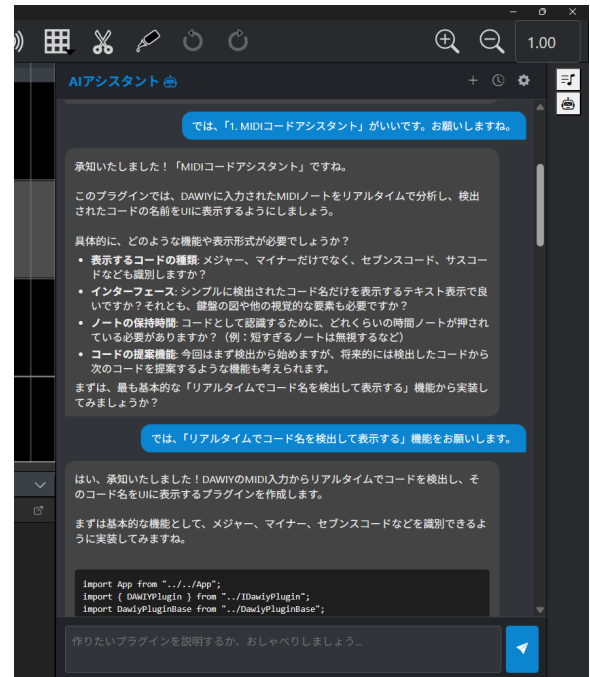


図 4: AI アシスタントのサイドバー

イルを作成したら, DAW 内部のクリエイターツール (図 3) にファイルをドラッグアンドドロップし, 名称, クラス名 (ID), グループ名, 説明等を記述してから生成ボタンを押すことで, プラグインを登録することが可能である.

4.3.1 AI アシスタント

DAWIY の画面右側に常駐する「AI アシスタント」(サイドバー) を実装した (図 4). AI アシスタントは, Google Gemini API や OpenAI API 等の LLM プロバイダと連携している. ユーザが自然言語で「リアルタイムにコード名を検出して表示する機能を作成して」「ランダムにメロディを生成する機能が欲しい」といった要望を入力すると, システムはプロンプトを LLM へ送信し, 要件定義から TypeScript コードの生成までを自動的に行う. 生成されたコードは DAWIY プラグインとして即座にビルド・ロードされる. この一連の流れにより, ユーザはコードを一切書くことなく, 対話をするだけで数十秒で独自の機能を DAWIY に追加することが可能となった.

4.4 事例

本研究では, DAWIY プラグインの有用性と表現力を検証するために, いくつかのプロトタイププラグインを実装した.

4.4.1 MIDI Chord Assistant

リアルタイムに MIDI 入力信号を解析し, 現在演奏されているコード (和音) の名前を表示する解析プラグインである. ユーザーが鍵盤などで和音を弾くと, その構成音から「C Major」や「A Minor 7」といったコード名を判定し, 専用のウィンドウに表示する. これにより, 演奏中や作曲

中の音楽理論的な補助を行うことができる。

4.4.2 Memo / Sticky Notes

トラックビュー上に、手書きのメモや付箋 (Sticky Note)、テキストボックスを自由に配置できる UI プラグインである。PIXI.js を利用したオーバーレイ描画により、楽曲の特定の箇所に対してアイデアや修正指示などを視覚的に残すことができる。通常の DAW のマーカー機能よりも自由度が高く、視覚的な情報整理に役立つ。

4.4.3 Stochastic Note Generator

確率的なアルゴリズムを用いて、ランダムなメロディやリズムパターンを生成する生成プラグインである。指定したスケールや確率分布に基づいてノートを生成し、トラックに配置する。作曲のアイデア出しや、偶然性を利用した音楽制作を支援する。

4.4.4 Sample IO & UI Plugin

独自のファイル形式 (.sampletext) の入出力を扱う機能と、サイドバーへのカスタム UI 追加を実証するためのプラグインである。テキストデータのインポート・エクスポート機能や、サイドバー内での独自コントロールの配置など、DAWIY の拡張性を網羅的に利用した実装例となっている。

5. 開発の現状と今後の展望

現在、DAWIY の基本機能となるピアノロール、オーディオ操作、およびプラグインシステムのコア部分は実装が完了しており、Web ブラウザ上で動作する DAW として最低限の機能を有している。特に、AI アシスタントを用いたプラグイン生成フローは、従来の DAW にはない革新的な機能として、ユーザによるカスタマイズの敷居を大きく下げること成功している。しかし、3 章で述べた機能のうち、カラーテーマの指定、ドラッグアンドドロップによる GUI の配置の移動などについては執筆時点で実装出来ていない。

今後の展望としては、以下の点が挙げられる。第一に、HostAPI の拡充である。現在プラグインから操作できる範囲は限定的であり、より高度な機能（詳細なミキサー操作、オートメーションの高度な制御など）を実現するためには、API の整備が必要である。第二に、AI アシスタントの精度向上である。複雑な要件に対するコード生成の正確性や、エラー発生時の自動修正機能などを強化し、より実用的な開発パートナーとしての能力を高めていく必要がある。第三に、プラグイン共有のエコシステムの構築である。作成したプラグインをユーザ間で容易に共有・評価できるプラットフォームを整備することで、DAWIY のコミュニティを活性化させ、多様な機能が自律的に生まれる環境を作ることが期待される。

参考文献

- [1] 承太郎山崎 : NumberZero-0621/DAWIY, <https://github.com/NumberZero-0621/DAWIY>.
- [2] Buffa, M.: WAM-studio, a Digital Audio Workstation (DAW) for the Web | Companion Proceedings of the ACM Web Conference 2023, <https://dl.acm.org/doi/abs/10.1145/3543873.3587987>.
- [3] Buffa, M., Ren, S., Campbell, O., Burns, T., Yi, S., Kleimola, J. and Larkin, O.: Web Audio Modules 2.0: An Open Web Audio Plugin Standard, *Companion Proceedings of the Web Conference 2022*, WWW '22, New York, NY, USA, Association for Computing Machinery, pp. 364–369 (online), DOI: 10.1145/3487553.3524225 (<https://dl.acm.org/doi/10.1145/3487553.3524225>).
- [4] K E N くん : AviUtil のお部屋, <https://spring-fragrance.mints.ne.jp/aviutil/>.